

JavaScript Prototypes

Introduction to Prototype-Based Programming

JavaScript implements object-oriented programming through a prototype-based model, which differs fundamentally from the class-based approach used in languages like C++, Java, or Python. Understanding prototypes is essential for mastering JavaScript's object system and writing efficient code.

What is a Prototype?

A prototype is an object that serves as a fallback source of properties and methods for other objects. When you try to access a property or method on an object, JavaScript first looks for it on the object itself. If it's not found, JavaScript searches the object's prototype, then the prototype's prototype, and so on, forming what's called the **prototype chain**.

Basic Object Relationships

Let's examine how objects can be linked through prototypes:

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal is walking");
  }
};

const dog = {
  barks: true,
  weight: 30
};

// Link dog to animal using setPrototypeOf
Object.setPrototypeOf(dog, animal);
```

```
console.log(dog.eats); // Accesses property from prototype  
dog.walk();           // Calls method from prototype
```

Output:

```
true  
Animal is walking
```

How the Prototype Chain Works

When `dog.eats` is accessed:

1. JavaScript looks for `eats` on the `dog` object itself
2. Not found, so it checks `dog` 's prototype (which is `animal`)
3. Finds `eats: true` on `animal` and returns it

When `dog.walk()` is called:

1. JavaScript looks for `walk` on the `dog` object
2. Not found, so it checks the prototype (`animal`)
3. Finds the `walk` method and executes it

 Note:

Multiple objects can share the same prototype, allowing efficient memory usage. All objects linked to the same prototype share its methods without duplicating them in memory.

Constructor Functions and Prototypes

The Problem: Duplicated Methods

Consider this constructor function:

```
function Person(name) {  
  this.name = name;  
  this.sayHi = function() {  
    console.log("Hi I am", this.name);  
  };  
}  
  
const p1 = new Person("Amit");  
const p2 = new Person("Aditi");  
  
console.log(p1.sayHi === p2.sayHi);
```

Output:

false

The output is `false` because each instance creates its own copy of the `sayHi` method. If you create 1000 Person objects, you'll have 1000 separate `sayHi` functions in memory, all doing the same thing.

⚠ Important:

The `===` operator checks both value and type (strict equality), while `==` checks only value with type coercion. Here, it compares whether both variables reference the exact same function object in memory.

The Solution: Prototype-Based Methods

Instead of defining methods inside the constructor, define them on the constructor's prototype:

```
function Person2(name) {  
  this.name = name;  
}  
  
// Define method on the prototype  
Person2.prototype.sayHi = function() {  
  console.log("Hi I am", this.name);  
};
```

```
const p3 = new Person2("Amit");  
const p4 = new Person2("Aditi");  
  
console.log(p3.sayHi === p4.sayHi);
```

Output:

true

Now both instances share the same `sayHi` method from `Person2.prototype`. This is memory-efficient: regardless of how many `Person2` instances you create, there's only one `sayHi` function in memory.

Property Shadowing

When an object has its own property with the same name as a prototype property, the object's own property takes precedence:

```
const animal = {  
  eats: true,  
  walk() {  
    console.log("Animal is walking");  
  }  
};  
  
const dog = {  
  barks: true,  
  weight: 30  
};  
  
Object.setPrototypeOf(dog, animal);  
  
// Modify the prototype  
animal.weight = 40;  
  
console.log(animal.weight); // From animal directly  
console.log(dog.weight);    // From dog's own property
```

Output:

40

30

The `dog` object has its own `weight` property (30), so it doesn't use the prototype's `weight` property (40). This is called **property shadowing**.

ES6 Classes: Syntactic Sugar Over Prototypes

ES6 introduced the `class` syntax, which provides a cleaner way to work with prototypes. However, it's important to understand that classes in JavaScript are not like classes in traditional OOP languages—they're syntactic sugar over the existing prototype system.

```
class PersonV2 {  
  constructor(name) {  
    this.name = name;  
  }  
  
  sayHi() {  
    console.log("Hi I am", this.name);  
  }  
}  
  
const p5 = new PersonV2("Amit");  
const p6 = new PersonV2("Aditi");  
  
console.log(p5.sayHi === p6.sayHi);  
p5.sayHi();
```

Output:

```
true  
Hi I am Amit
```

Behind the scenes, this class creates:

- A constructor function named `PersonV2`
- A `sayHi` method on `PersonV2.prototype`

 Note:

Classes are strict mode by default, meaning they enforce stricter parsing and error handling. This includes restrictions like not being able to use undeclared variables and not allowing duplicate parameter names.

Comparison: Traditional OOP vs JavaScript Prototypes

Traditional OOP (C++, Java)	JavaScript Prototypes
Each instance has its own copy of methods	Instances share methods through prototypes
Classes are blueprints compiled before runtime	Prototypes are live objects that can be modified at runtime
Inheritance through class hierarchies	Inheritance through prototype chains
Memory usage: Higher (methods duplicated)	Memory usage: Lower (methods shared)

Memory Efficiency Example

Consider creating 1000 employee objects with methods `setData` and `printData`: 

Without Prototypes: 1000 objects × 2 methods = 2000 function objects in memory

With Prototypes: 1000 objects + 2 shared methods = 2 function objects in memory

This demonstrates the significant memory advantage of the prototype approach.

Key Takeaways

- JavaScript uses prototype-based inheritance, not class-based inheritance
- Prototypes enable memory-efficient sharing of methods across objects
- The prototype chain allows objects to inherit properties and methods
- Constructor functions use `prototype` property to define shared methods

- ES6 classes are syntactic sugar that still use prototypes underneath
- Property shadowing occurs when an object has its own property with the same name as a prototype property
- JavaScript is multi-paradigm: it supports object-oriented, functional, and imperative programming styles

Best Practices

1. **Define methods on prototypes**, not inside constructors, to save memory
2. **Use ES6 class syntax** for cleaner, more readable code while understanding the underlying prototype mechanism
3. **Avoid modifying built-in prototypes** (like `Array.prototype` or `Object.prototype`) as it can lead to unexpected behavior
4. **Use `Object.setPrototypeOf` sparingly** in production code as it can impact performance; prefer `Object.create` or classes instead

Further Exploration

Now that you understand the fundamentals, let's explore additional prototype-related concepts with practical examples.

1. `Object.create()` - A Better Way to Set Prototypes

While `Object.setPrototypeOf()` works, `Object.create()` is the preferred method for creating objects with specific prototypes because it's more performant.

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal is walking");
  }
};

// Create a new object with animal as its prototype
const cat = Object.create(animal);
cat.meows = true;
cat.name = "Whiskers";
```

```
console.log(cat.eats);      // From prototype
console.log(cat.meows);    // Own property
cat.walk();                // From prototype
console.log(cat.name);     // Own property
```

Output:

```
true
true
Animal is walking
Whiskers
```

`Object.create(Animal)` creates a new empty object and sets `Animal` as its prototype in one operation, which is more efficient than creating an object and then modifying its prototype.

2. instanceof Operator - Checking Prototype Relationships

The `instanceof` operator checks whether an object has a specific constructor's prototype anywhere in its prototype chain.

```
function Dog(name) {
  this.name = name;
}

Dog.prototype.bark = function() {
  console.log(this.name + " says Woof!");
};

const myDog = new Dog("Buddy");

console.log(myDog instanceof Dog);    // Is myDog created by Dog?
console.log(myDog instanceof Object); // Is myDog an Object?
console.log(myDog instanceof Array);  // Is myDog an Array?
```

Output:


```
true  
true  
false
```

This works because:

- `myDog` was created with `new Dog()`, so `Dog.prototype` is in its chain
- All objects in JavaScript inherit from `Object.prototype`, so all objects are instances of `Object`
- `myDog` was not created as an array, so it's not an instance of `Array`

3. `hasOwnProperty()` - Distinguishing Own vs Inherited Properties

This method helps you determine if a property belongs directly to an object or is inherited from its prototype.

```
function Car(brand) {  
  this.brand = brand;  
}  
  
Car.prototype.wheels = 4;  
  
const myCar = new Car("Toyota");  
  
console.log(myCar.brand);           // Own property  
console.log(myCar.wheels);          // Inherited property  
  
console.log(myCar.hasOwnProperty('brand')); // Check if own  
console.log(myCar.hasOwnProperty('wheels')); // Check if own
```

Output:

```
Toyota  
4  
true  
false
```

This is particularly useful when iterating over object properties, as you can filter out inherited properties:

```
for (let key in myCar) {  
  if (myCar.hasOwnProperty(key)) {  
    console.log(key + ": " + myCar[key]);  
  }  
}
```

Output:

brand: Toyota

4. Prototype Chain Traversal

Every prototype chain eventually ends at `Object.prototype`, which is the root prototype of all JavaScript objects. Beyond that is `null`.

```
function Animal(name) {  
  this.name = name;  
}  
  
const dog = new Animal("Rex");  
  
// Get the prototype of dog  
console.log(Object.getPrototypeOf(dog) === Animal.prototype);  
  
// Get the prototype of Animal.prototype  
console.log(Object.getPrototypeOf(Animal.prototype) === Object.prototype);  
  
// Get the prototype of Object.prototype  
console.log(Object.getPrototypeOf(Object.prototype));
```

Output:

true
true
null

The complete chain for `dog` is:

`dog` → `Animal.prototype` → `Object.prototype` → `null`

 Note:

When JavaScript can't find a property anywhere in the prototype chain and reaches `null`, it returns `undefined`.

5. Modern Class Features

Class Inheritance with extends

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name + " makes a sound");
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // Call parent constructor
    this.breed = breed;
  }

  speak() {
    console.log(this.name + " barks");
  }
}

const myDog = new Dog("Max", "Golden Retriever");
myDog.speak();
console.log(myDog.name);
console.log(myDog.breed);
```

Output:

Max barks
Max
Golden Retriever

Static Methods

Static methods belong to the class itself, not to instances of the class.

```
class MathHelper {  
    static add(a, b) {  
        return a + b;  
    }  
  
    static multiply(a, b) {  
        return a * b;  
    }  
}  
  
// Call static method on the class itself  
console.log(MathHelper.add(5, 3));  
console.log(MathHelper.multiply(4, 7));  
  
// Cannot call on instances  
const helper = new MathHelper();  
// helper.add(2, 3); // This would cause an error!
```

Output:

8
28

Using super in Methods

The `super` keyword allows you to call methods from the parent class.

```
class Animal {  
    constructor(name) {  
        this.name = name;  
    }  
}
```

```
    describe() {
        console.log("This is " + this.name);
    }
}

class Bird extends Animal {
    constructor(name, canFly) {
        super(name);
        this.canFly = canFly;
    }

    describe() {
        super.describe(); // Call parent method
        console.log(this.name + " can" + (this.canFly ? "" : "not") + " fly");
    }
}

const eagle = new Bird("Eagle", true);
eagle.describe();
```

Output:

This is Eagle
Eagle can fly

Private Fields (Modern JavaScript)

Private fields are prefixed with `#` and can only be accessed within the class.

```
class BankAccount {
    #balance = 0; // Private field

    constructor(initialBalance) {
        this.#balance = initialBalance;
    }

    deposit(amount) {
        this.#balance += amount;
        console.log("Deposited: $" + amount);
    }
}
```

```
getBalance() {  
    return this.#balance;  
}  
}  
  
const account = new BankAccount(100);  
account.deposit(50);  
console.log("Balance: $" + account.getBalance());  
  
// console.log(account.#balance); // This would cause an error!
```

Output:

Deposited: \$50
Balance: \$150

Note:

Private fields provide true encapsulation, ensuring that certain data can only be accessed and modified through designated methods, which is a core principle of object-oriented programming.

Practical Example: Combining Concepts

Let's create a more complex example that uses multiple concepts together:

```
class Vehicle {  
    #mileage = 0;  
  
    constructor(brand, model) {  
        this.brand = brand;  
        this.model = model;  
    }  
  
    drive(miles) {  
        this.#mileage += miles;  
        console.log(`Drove ${miles} miles`);  
    }  
  
    getMileage() {  
        return this.#mileage;  
    }  
}
```

```
}

static compare(vehicle1, vehicle2) {
  const diff = vehicle1.getMileage() - vehicle2.getMileage();
  return diff > 0 ? vehicle1 : vehicle2;
}

}

class ElectricCar extends Vehicle {
  constructor(brand, model, batteryCapacity) {
    super(brand, model);
    this.batteryCapacity = batteryCapacity;
  }

  charge() {
    console.log(`Charging ${this.brand} ${this.model}`);
  }
}

const car1 = new ElectricCar("Tesla", "Model 3", 75);
const car2 = new ElectricCar("Nissan", "Leaf", 62);

car1.drive(150);
car2.drive(200);

console.log(`${car1.brand} mileage: ${car1.getMileage()}`);
console.log(`${car2.brand} mileage: ${car2.getMileage()}`);

const moreDriven = Vehicle.compare(car1, car2);
console.log(`More driven: ${moreDriven.brand} ${moreDriven.model}`);
```

Output:

Drove 150 miles
Drove 200 miles
Tesla mileage: 150
Nissan mileage: 200
More driven: Nissan Leaf

This example demonstrates:

- Private fields (`#mileage`) for encapsulation

- Class inheritance (`ElectricCar` extends `Vehicle`)
- The `super` keyword for calling parent constructors
- Static methods (`Vehicle.compare`) for class-level operations
- Method inheritance and the prototype chain working behind the scenes

